JavaScript Course - YouLearn

Project-Based, Hands-On (Beginner Syntax → Confident Builder)
For: A strong Python coder (PCAP level) who is new to JavaScript syntax, age 14-friendly
Outcome: You will build **TaskFlow**, a real task manager app, and finish able to code JavaScript confidently (roughly "PCAP-equivalent" skill: core syntax, functions, data structures, debugging, async, DOM, and a React starter).

How to Use This Course

- You will build **one project** the whole time: **TaskFlow**.
- Each section adds features to the same codebase.
- Do the **Checkpoint** before moving on.

Tools & Setup (Do this once)

What you need

- A folder named `taskflow/`
- Inside it, create:
  - `index.html`
- `style.css` (optional at first)
- `app.js`

The simplest "JavaScript runs in a page" starter

index.html

```html
```

```
<!doctype html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>TaskFlow</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <h1>TaskFlow</h1>

    <script src="app.js"></script>
  </body>
</html>
```

app.js

```js
console.log("TaskFlow loaded!");
```

Checkpoint

- Open `index.html` in your browser.
- Press F12 → Console → you should see: `TaskFlow loaded!`

Project: TaskFlow (Build a Task Manager)

Data Model (used throughout the course)

A **task** is a plain object:

- `id` (number)
- `title` (string)
- `done` (boolean)

Example:

```js
```

```
{ id: 123, title: "Learn JS", done: false }
```

Section 1 — JavaScript Syntax Basics (Variables, Types, Comments)

Goal

Write correct JS syntax for variables and understand basic types.

What you'll build

A starting task list stored in JavaScript.

Core syntax

```js
// Comments
// This is a single-line comment

const appName = "TaskFlow"; // string
let version = 1;            // number
const isReady = true;       // boolean

let emptyValue = null;      // intentional empty
let notAssigned;            // undefined (declared, not set)
```

Project step

Create your tasks array:

```js
const tasks = [
  { id: 1, title: "Install my project folder", done: true },
  { id: 2, title: "Learn JS variables", done: false },
  { id: 3, title: "Build TaskFlow UI", done: false }
];

console.log(tasks);
```

Common mistakes

- Using `var` (skip it for now; use `const` / `let`).
- Forgetting commas in arrays/objects.

Checkpoint

Add 2 more tasks to the array and confirm they print in the console.

Section 2 — Operators, Conditions, and Loops (If, For, For..of)

Goal

Control program flow like you do in Python, but in JS syntax.

Core rules (important)

Use **strict equality**:

- Use `===` and `!==`
- Avoid `==` (causes confusing type coercion)

Project step: print tasks nicely

```js
function printTasks(taskArray) {
  for (const task of taskArray) {
    if (task.done === true) {
      console.log("[DONE]", task.title);
    } else {
      console.log("[TODO]", task.title);
    }
  }
}

printTasks(tasks);
```

Common mistakes

- Writing `if (x = 3)` instead of `if (x === 3)`
- Forgetting curly braces in multi-line `if` blocks

Checkpoint

Change one task's `done` to `true` and confirm the output updates.

Section 3 — Functions (Declarations, Arrow Functions, Return)

Goal

Write reusable functions (the same way you think in Python).

Core syntax

Regular function

```js
function add(a, b) {
  return a + b;
}
```

Arrow function

```js
const add2 = (a, b) => a + b;
```

Project step: implement core task actions

```js
```

```
function addTask(title) {
  const newTask = {
    id: Date.now(),
    title: title,
    done: false
  };

  tasks.push(newTask);
  return newTask;
}

function toggleTaskDone(id) {
  for (const task of tasks) {
    if (task.id === id) {
      task.done = !task.done;
      return task;
    }
  }
  return null;
}
```

Common mistakes

- Forgetting `return`
- Mixing up `()` vs `{}` in arrow functions

Checkpoint

Call `addTask("Practice JS functions")`, then toggle it to done.

Section 4 — Arrays & Objects (Map, Filter, Find)

Goal

Use the most common “real-world JS” data tools.

Project step: searching and filtering

```js
```

```
function getDoneTasks() {
  return tasks.filter(t => t.done === true);
}

function findTaskById(id) {
  return tasks.find(t => t.id === id);
}

function getTitles() {
  return tasks.map(t => t.title);
}
```

Common mistakes

- Confusing `find` (returns one item) with `filter` (returns many)
- Forgetting arrow function `=>`

Checkpoint

Log:

- the titles array
- only done tasks
- one task found by id

Section 5 — HTML + DOM Basics (Make It Visible)

Goal

Render tasks onto the page, not just the console.

Project step: update HTML

index.html (inside `<body>`, under the `<h1>`)

```html
```

```
<div id="controls">
  <input id="taskInput" placeholder="New task..." />
  <button id="addBtn">Add</button>
</div>

<ul id="taskList"></ul>
```

Project step: render tasks

app.js

```js
const taskListEl = document.querySelector("#taskList");
const taskInputEl = document.querySelector("#taskInput");
const addBtnEl = document.querySelector("#addBtn");

function renderTasks() {
  taskListEl.innerHTML = "";

  for (const task of tasks) {
    const li = document.createElement("li");
    li.textContent = task.done ? `✅ ${task.title}` : `⬜ ${task.title}`;
    taskListEl.appendChild(li);
  }
}

renderTasks();
```

Common mistakes

- Running JS before the HTML exists (fix: keep `<script src="app.js"></script>` at the end of `<body>`)
- Forgetting `#` in `querySelector("#id")`

Checkpoint

When you refresh the page, you should see your tasks displayed.

Section 6 — Events (Buttons, Input, Updating UI)

Goal

Make the app interactive: add tasks from the input.

Project step: add click event

```js
addBtnEl.addEventListener("click", () => {
  const title = taskInputEl.value.trim();
  if (title.length === 0) return;

  addTask(title);
  taskInputEl.value = "";
  renderTasks();
});
```

Common mistakes

- Forgetting `.trim()` and allowing empty tasks
- Updating data but forgetting to call `renderTasks()`

Checkpoint

You can add tasks from the UI.

Section 7 — Editing & Deleting Tasks (Buttons Per Task)

Goal

Add “Delete” and “Toggle Done” buttons for each task.

Project step: render buttons with safe events

```js
```

```
function deleteTask(id) {
  const index = tasks.findIndex(t => t.id === id);
  if (index !== -1) tasks.splice(index, 1);
}

function renderTasks() {
  taskListEl.innerHTML = "";

  for (const task of tasks) {
    const li = document.createElement("li");

    const titleSpan = document.createElement("span");
    titleSpan.textContent = task.done ? `✅ ${task.title}` : `⬜ ${task.title}`;

    const toggleBtn = document.createElement("button");
    toggleBtn.textContent = "Toggle";
    toggleBtn.addEventListener("click", () => {
      toggleTaskDone(task.id);
      renderTasks();
    });

    const delBtn = document.createElement("button");
    delBtn.textContent = "Delete";
    delBtn.addEventListener("click", () => {
      deleteTask(task.id);
      renderTasks();
    });

    li.appendChild(titleSpan);
    li.appendChild(toggleBtn);
    li.appendChild(delBtn);

    taskListEl.appendChild(li);
  }
}
```

Common mistakes

- Using `innerHTML` to inject buttons and then struggling with event handling
- Forgetting that `Date.now()` ids are numbers (compare as numbers)

Checkpoint

You can toggle and delete tasks from the UI.

Section 8 — Persistence with localStorage (Save on Refresh)

Goal

Make tasks stay after refreshing the page.

Project step

```js
function saveTasks() {
  localStorage.setItem("taskflow_tasks", JSON.stringify(tasks));
}

function loadTasks() {
  const raw = localStorage.getItem("taskflow_tasks");
  if (!raw) return;

  const loaded = JSON.parse(raw);
  tasks.length = 0;
  tasks.push(...loaded);
}
```

Use it like this:

```js
loadTasks();
renderTasks();
```

And after every change:

```js
addTask(title);
saveTasks();
renderTasks();
```

Common mistakes

- Forgetting `JSON.stringify` / `JSON.parse`
- Not saving after toggle/delete

Checkpoint

Refresh the page: tasks should still be there.

Section 9 — Async JavaScript + Fetch (Headless API Integration)

Goal

Learn `async/await` syntax and connect your frontend directly to an API layer.

Core pattern

```js
async function getTasksFromApi() {
  const res = await fetch("http://localhost:8000/tasks");
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return await res.json();
}
```

Project step: swap localStorage for API calls (basic version)

```js
async function reloadFromApi() {
  const apiTasks = await getTasksFromApi();
  tasks.length = 0;
  tasks.push(...apiTasks);
  renderTasks();
}
```

Common mistakes

- Forgetting `await` (you'll get Promises instead of data)
- Not handling non-OK responses
- CORS issues (backend must allow your frontend origin)

Checkpoint

A "Load from API" button loads tasks from your backend and renders them.

Section 10 — React Upgrade Path (After You Own the Syntax)

Goal

Move from manual DOM rendering to React's state-driven UI.

What you'll rebuild

TaskFlow UI in React using:

- `useState` for tasks
- `useEffect` for initial load
- `.map()` to render list
- fetch calls in async functions

Checkpoint

You can explain:

- Vanilla: "I update the DOM myself"
- React: "I update state; React updates the UI"

Final Skill Checklist (PCAP-Equivalent Target)

You can now:

- Write JS syntax confidently (`const/let`, functions, loops, objects, arrays)
- Use strict comparisons (`===`)
- Build a dynamic UI with DOM + events
- Persist data with localStorage
- Use `async/await` + `fetch()` to talk to APIs (headless style)
- Transition into React without being confused by syntax